

template.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define rep(i, n)
4 using ll = long long;
5
6 void solve() {
7 }
8
9 signed main() {
10  cin.tie(0);
11  ios::sync_with_stdio(0);
12  cout << fixed << setprecision(20);
13  int TT;
14  TT = 1; //cin >> TT;
15  solve();
16 }

```

dsu.cpp

```

1 struct dsu {
2  vector<int> p;
3  dsu(int n) : p(n, -1) {}
4  int leader(int x) { return p[x] < 0 ? x : p[x] = leader(p[x]); }
5  int merge(int a, int b) {
6    a = leader(a);
7    b = leader(b);
8    if (a == b)
9      return a;
10   if (p[a] > p[b])
11     swap(a, b);
12   p[a] += p[b];
13   p[b] = a;
14   return a;
15 }
16 bool same(int a, int b) { return leader(a) == leader(b); }
17 int size(int a) { return -p[leader(a)]; }
18 };

```

segtree.cpp

```

1 template <class S, S op(S, S), S e()>
2 struct segtree {
3  int n, sz;
4  vector<S> d;
5  segtree(int m = 0) : n(1), sz(m) {
6    while (n < m)
7      n *= 2;

```

segtree.cpp (cont.)

```

8    d.assign(2 * n, e());
9  }
10 segtree(vector<S> a) : segtree(a.size()) {
11  copy(a.begin(), a.end(), d.begin() + n);
12  for (int i = n - 1; i; i--)
13    d[i] = op(d[2 * i], d[2 * i + 1]);
14 }
15 void set(int p, S x) {
16  for (d[p += n] = x; p /= 2; )
17    d[p] = op(d[2 * p], d[2 * p + 1]);
18 }
19 S get(int p) { return d[p + n]; }
20 S prod(int l, int r) {
21  S x = e(), y = e();
22  for (l += n, r += n; l < r; l /= 2, r /= 2) {
23    if (l & 1)
24      x = op(x, d[l++]);
25    if (r & 1)
26      y = op(d[--r], y);
27  }
28  return op(x, y);
29 }
30 S all_prod() { return d[1]; }
31
32 template <class G>
33 int max_right(int l, G g) {
34  assert(g(e()));
35  S x = e();
36  int r = mr(l, g, x, 1, 0, n);
37  return r < 0 ? sz : r;
38 }
39 template <class G>
40 int mr(int a, G& g, S& x, int k, int l, int r) {
41  if (r <= a || sz <= l)
42    return -1;
43  if (a <= l && r <= sz) {
44    S y = op(x, d[k]);
45    if (g(y)) {
46      x = y;
47      return -1;
48    }
49    if (r - l == 1)
50      return l;
51  }
52  int m = (l + r) / 2, q = mr(a, g, x, 2 * k, l, m);

```

segtree.cpp (cont.)

```

53     return q < 0 ? mr(a, g, x, 2 * k + 1, m, r) : q;
54 }
55 template <class G>
56 int min_left(int r, G g) {
57     assert(g(e()));
58     S x = e();
59     int l = ml(r, g, x, 1, 0, n);
60     return l < 0 ? 0 : l;
61 }
62 template <class G>
63 int ml(int b, G& g, S& x, int k, int l, int r) {
64     if (b <= l)
65         return -1;
66     if (r <= b) {
67         S y = op(d[k], x);
68         if (g(y)) {
69             x = y;
70             return -1;
71         }
72         if (r - l == 1)
73             return r;
74     }
75     int m = (l + r) / 2, q = ml(b, g, x, 2 * k + 1, m, r);
76     return q < 0 ? ml(b, g, x, 2 * k, l, m) : q;
77 }
78 };

```

lazy_segtree.cpp

```

1 template <class S, S op(S, S), S e(), class F, S mapping(F, S),
2   F composition(F, F), F id()>
3 struct lazy_segtree {
4     int n, sz;
5     vector<S> d;
6     vector<F> lz;
7     lazy_segtree(int m = 0) : n(1), sz(m) {
8         while (n < m)
9             n *= 2;
10        d.assign(2 * n, e());
11        lz.assign(2 * n, id());
12    }
13    lazy_segtree(vector<S> a) : lazy_segtree(a.size()) {
14        copy(a.begin(), a.end(), d.begin() + n);
15        for (int i = n - 1; i; i--)
16            d[i] = op(d[2 * i], d[2 * i + 1]);
17    }

```

lazy_segtree.cpp (cont.)

```

18 void all(int k, F f) {
19     d[k] = mapping(f, d[k]);
20     lz[k] = composition(f, lz[k]);
21 }
22 void push(int k) {
23     all(2 * k, lz[k]);
24     all(2 * k + 1, lz[k]);
25     lz[k] = id();
26 }
27 void set(int p, S x) { set(p, x, 1, 0, n); }
28 void set(int p, S x, int k, int l, int r) {
29     if (r - l == 1) {
30         d[k] = x;
31         lz[k] = id();
32         return;
33     }
34     push(k);
35     int m = (l + r) / 2;
36     p < m ? set(p, x, 2 * k, l, m) : set(p, x, 2 * k + 1, m, r);
37     d[k] = op(d[2 * k], d[2 * k + 1]);
38 }
39 S get(int p) { return prod(p, p + 1); }
40 S prod(int a, int b) { return prod(a, b, 1, 0, n); }
41 S prod(int a, int b, int k, int l, int r) {
42     if (r <= a || b <= l)
43         return e();
44     if (a <= l && r <= b)
45         return d[k];
46     push(k);
47     int m = (l + r) / 2;
48     return op(
49         prod(a, b, 2 * k, l, m), prod(a, b, 2 * k + 1, m, r));
50 }
51 S all_prod() { return d[1]; }
52 void apply(int p, F f) { apply(p, p + 1, f); }
53 void apply(int a, int b, F f) { apply(a, b, f, 1, 0, n); }
54 void apply(int a, int b, F f, int k, int l, int r) {
55     if (r <= a || b <= l)
56         return;
57     if (a <= l && r <= b) {
58         all(k, f);
59         return;
60     }
61     push(k);
62     int m = (l + r) / 2;

```

lazy_segtree.cpp (cont.)

```

63  apply(a, b, f, 2 * k, l, m);
64  apply(a, b, f, 2 * k + 1, m, r);
65  d[k] = op(d[2 * k], d[2 * k + 1]);
66  }
67
68  template <class G>
69  int max_right(int l, G g) {
70      assert(g(e()));
71      S x = e();
72      int r = mr(l, g, x, 1, 0, n);
73      return r < 0 ? sz : r;
74  }
75  template <class G>
76  int mr(int a, G& g, S& x, int k, int l, int r) {
77      if (r <= a || sz <= l)
78          return -1;
79      if (a <= l && r <= sz) {
80          S y = op(x, d[k]);
81          if (g(y)) {
82              x = y;
83              return -1;
84          }
85          if (r - l == 1)
86              return l;
87      }
88      push(k);
89      int m = (l + r) / 2;
90      int q = mr(a, g, x, 2 * k, l, m);
91      return q < 0 ? mr(a, g, x, 2 * k + 1, m, r) : q;
92  }
93  template <class G>
94  int min_left(int r, G g) {
95      assert(g(e()));
96      S x = e();
97      int l = ml(r, g, x, 1, 0, n);
98      return l < 0 ? 0 : l;
99  }
100 template <class G>
101 int ml(int b, G& g, S& x, int k, int l, int r) {
102     if (b <= l)
103         return -1;
104     if (r <= b) {
105         S y = op(d[k], x);

```

lazy_segtree.cpp (cont.)

```

106     if (g(y)) {
107         x = y;
108         return -1;
109     }
110     if (r - l == 1)
111         return r;
112 }
113 push(k);
114 int m = (l + r) / 2;
115 int q = ml(b, g, x, 2 * k + 1, m, r);
116 return q < 0 ? ml(b, g, x, 2 * k, l, m) : q;
117 }
118 };
119
120 template <class S, S op(S, S), S e()>
121 struct segtree {
122     int n, sz;
123     vector<S> d;
124     segtree(int m = 0) : n(1), sz(m) {
125         while (n < m)
126             n *= 2;
127         d.assign(2 * n, e());
128     }
129     segtree(vector<S> a) : segtree(a.size()) {
130         copy(a.begin(), a.end(), d.begin() + n);
131         for (int i = n - 1; i; i--)
132             d[i] = op(d[2 * i], d[2 * i + 1]);
133     }
134     void set(int p, S x) {
135         for (d[p += n] = x; p /= 2;)
136             d[p] = op(d[2 * p], d[2 * p + 1]);
137     }
138     S get(int p) { return d[p + n]; }
139     S prod(int l, int r) {
140         S x = e(), y = e();
141         for (l += n, r += n; l < r; l /= 2, r /= 2) {
142             if (l & 1)
143                 x = op(x, d[l++]);
144             if (r & 1)
145                 y = op(d[--r], y);
146         }
147         return op(x, y);
148     }
149     S all_prod() { return d[1]; }
150 }

```

lazy_segtree.cpp (cont.)

```

151 template <class G>
152 int max_right(int l, G g) {
153     assert(g(e()));
154     S x = e();
155     int r = mr(l, g, x, 1, 0, n);
156     return r < 0 ? sz : r;
157 }
158 template <class G>
159 int mr(int a, G& g, S& x, int k, int l, int r) {
160     if (r <= a || sz <= l)
161         return -1;
162     if (a <= l && r <= sz) {
163         S y = op(x, d[k]);
164         if (g(y)) {
165             x = y;
166             return -1;
167         }
168         if (r - l == 1)
169             return l;
170     }
171     int m = (l + r) / 2, q = mr(a, g, x, 2 * k, l, m);
172     return q < 0 ? mr(a, g, x, 2 * k + 1, m, r) : q;
173 }
174 template <class G>
175 int min_left(int r, G g) {
176     assert(g(e()));
177     S x = e();
178     int l = ml(r, g, x, 1, 0, n);
179     return l < 0 ? 0 : l;
180 }
181 template <class G>
182 int ml(int b, G& g, S& x, int k, int l, int r) {
183     if (b <= l)
184         return -1;
185     if (r <= b) {
186         S y = op(d[k], x);
187         if (g(y)) {
188             x = y;
189             return -1;
190         }
191         if (r - l == 1)
192             return r;
193     }
194     int m = (l + r) / 2, q = ml(b, g, x, 2 * k + 1, m, r);

```

lazy_segtree.cpp (cont.)

```

195     return q < 0 ? ml(b, g, x, 2 * k, l, m) : q;
196 }
197 };

```

binarytrie.cpp

```

1 template <int B = 30>
2 struct binary_trie {
3     struct N {
4         int nx[2] = {-1, -1}, cnt = 0;
5     };
6     vector<N> t{N()};
7     void add(long long x, int d = 1) {
8         int v = 0;
9         t[v].cnt += d;
10        for (int i = B - 1; i >= 0; i--) {
11            int b = x >> i & 1;
12            if (t[v].nx[b] < 0)
13                t[v].nx[b] = t.size(), t.emplace_back();
14            v = t[v].nx[b];
15            t[v].cnt += d;
16        }
17    }
18    void insert(long long x) { add(x, 1); }
19    void erase(long long x) { add(x, -1); }
20    long long min_xor(long long x) {
21        int v = 0;
22        long long r = 0;
23        for (int i = B - 1; i >= 0; i--) {
24            int b = x >> i & 1, u = t[v].nx[b];
25            if (u < 0 || !t[u].cnt)
26                b ^= 1, r |= 1LL << i;
27            v = t[v].nx[b];
28        }
29        return r;
30    }
31    long long max_xor(long long x) {
32        int v = 0;
33        long long r = 0;
34        for (int i = B - 1; i >= 0; i--) {
35            int b = x >> i & 1, u = t[v].nx[b ^ 1];
36            if (u >= 0 && t[u].cnt)
37                b ^= 1, r |= 1LL << i;
38            v = t[v].nx[b];
39        }

```

binarytrie.cpp (cont.)

```

40     return r;
41 }
42 };

```

trie.cpp

```

1  template <int K, char B>
2  struct trie { //文字種類数 / 最初の文字
3      struct N {
4          int nx[K], end = 0;
5          N() { fill(nx, nx + K, -1); }
6      };
7      vector<N> t{N()};
8      void insert(string s) {
9          int v = 0;
10         for (char c : s) {
11             int x = c - B; //x が id にあたる 連続でないなら id(c)に変更
12             if (t[v].nx[x] < 0)
13                 t[v].nx[x] = t.size(), t.emplace_back();
14             v = t[v].nx[x];
15         }
16         t[v].end++;
17     }
18     int find(string s) {
19         int v = 0;
20         for (char c : s) {
21             v = t[v].nx[c - B];
22             if (v < 0)
23                 return -1;
24         }
25         return v;
26     }
27 };

```

suffix_array.cpp

```

1  template <class T>
2  vector<int> suffix_array(vector<T> s) {
3      int n = s.size();
4      if (!n)
5          return {};
6      int N = n + 1;
7      vector<int> p(N), c(N), q(N), nc(N), cnt(N);
8      p[0] = n;
9      iota(p.begin() + 1, p.end(), 0);
10     sort(p.begin() + 1, p.end(),
11         [&](int i, int j) { return s[i] < s[j]; });

```

suffix_array.cpp (cont.)

```

12     for (int i = 1; i < N; i++)
13         c[p[i]] = i == 1 ? 1 : c[p[i - 1]] + (s[p[i - 1]] != s[p[i]]);
14     for (int k = 1; k < N; k *= 2) {
15         for (int i = 0; i < N; i++)
16             q[i] = (p[i] - k + N) % N;
17         fill(cnt.begin(), cnt.end(), 0);
18         for (int x : q)
19             cnt[c[x]]++;
20         for (int i = 1; i < N; i++)
21             cnt[i] += cnt[i - 1];
22         for (int i = N - 1; i >= 0; i--)
23             p[--cnt[c[q[i]]]] = q[i];
24         nc[p[0]] = 0;
25         for (int i = 1; i < N; i++)
26             nc[p[i]] = nc[p[i - 1]] +
27                 (c[p[i - 1]] != c[p[i]] ||
28                  c[(p[i - 1] + k) % N] != c[(p[i] + k) % N]);
29         c.swap(nc);
30     }
31     return {p.begin() + 1, p.end()};
32 }
33 vector<int> suffix_array(string s) {
34     return suffix_array(vector<unsigned char>(s.begin(), s.end()));
35 }
36 vector<int> suffix_array(vector<int> s, int) {
37     return suffix_array<int>(move(s));
38 }

```

lcp_array.cpp

```

1  template <class T>
2  vector<int> lcp_array(const T& s, const vector<int>& sa) {
3      int n = s.size(), h = 0;
4      vector<int> r(n), l(n - 1);
5      for (int i = 0; i < n; i++)
6          r[sa[i]] = i;
7      for (int i = 0; i < n; i++)
8          if (r[i]) {
9              int j = sa[r[i] - 1];
10             while (max(i, j) + h < n && s[i + h] == s[j + h])
11                 h++;
12             l[r[i] - 1] = h;
13             if (h)
14                 h--;
15         }

```

lcp_array.cpp (cont.)

```
16 return l;
17 }
```

z_algorithm.cpp

```
1 template <class T>
2 vector<int> z_algorithm(const T& s) {
3     int n = s.size();
4     vector<int> z(n);
5     for (int i = 1, j = 0; i < n; i++) {
6         if (i < j + z[j])
7             z[i] = min(j + z[j] - i, z[i - j]);
8         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
9             z[i]++;
10        if (i + z[i] > j + z[j])
11            j = i;
12    }
13    if (n)
14        z[0] = n;
15    return z;
16 }
```

rolling_hash.cpp

```
1 struct rolling_hash {
2     using ull = unsigned long long;
3     using u128 = __uint128_t;
4     static constexpr ull M = (1ULL << 61) - 1;
5     static ull mul(ull a, ull b) {
6         u128 x = (u128)a * b;
7         ull y = (x >> 61) + (x & M);
8         return y >= M ? y - M : y;
9     }
10    inline static ull B =
11        chrono::steady_clock::now().time_since_epoch().count() %
12        (M - 256) +
13        256;
14    vector<ull> h, p;
15    rolling_hash(const string& s)
16        : h(s.size() + 1), p(s.size() + 1, 1) {
17        for (int i = 0; i < (int)s.size(); i++)
18            p[i + 1] = mul(p[i], B),
19            h[i + 1] =
20                (mul(h[i], B) + (unsigned char)s[i] + 1) % M;
21    }
```

rolling_hash.cpp (cont.)

```
22 ull get(int l, int r) const {
23     ull x = h[r] + M - mul(h[l], p[r - l]);
24     return x >= M ? x - M : x;
25 }
26 };
```

scc.cpp

```
1 struct scc_graph {
2     int n;
3     vector<vector<int>> g, rg;
4     scc_graph(int n) : n(n), g(n), rg(n) {}
5     void add_edge(int a, int b) {
6         g[a].push_back(b);
7         rg[b].push_back(a);
8     }
9     vector<vector<int>> scc() {
10        vector<int> vis(n), ord;
11        for (int s = 0; s < n; s++)
12            if (!vis[s]) {
13                stack<pair<int, int>> q;
14                q.push({s, 0});
15                vis[s] = 1;
16                while (q.size()) {
17                    auto& [v, i] = q.top();
18                    if (i < (int)g[v].size()) {
19                        int u = g[v][i++];
20                        if (!vis[u])
21                            vis[u] = 1, q.push({u, 0});
22                    } else
23                        ord.push_back(v), q.pop();
24                }
25            }
26        fill(vis.begin(), vis.end(), 0);
27        reverse(ord.begin(), ord.end());
28        vector<vector<int>> res;
29        for (int s : ord)
30            if (!vis[s]) {
31                res.push_back({});
32                stack<int> q;
33                q.push(s);
34                vis[s] = 1;
35                while (q.size()) {
36                    int v = q.top();
37                    q.pop();
38                    res.back().push_back(v);
```

scc.cpp (cont.)

```

39     for (int u : rg[v])
40         if (!vis[u])
41             vis[u] = 1, q.push(u);
42     }
43 }
44 return res;
45 }
46 };

```

twosat.cpp

```

1 struct two_sat {
2     int n;
3     vector<vector<int>> g, rg;
4     vector<bool> a;
5     two_sat(int N) : n(N), g(2 * N), rg(2 * N), a(N) {}
6
7     void add(int x, int y) {
8         g[x].push_back(y);
9         rg[y].push_back(x);
10    }
11    void add_clause(int i, bool f, int j, bool h) {
12        add(2 * i + !f, 2 * j + h);
13        add(2 * j + !h, 2 * i + f);
14    }
15
16    bool satisfiable() {
17        int N = 2 * n;
18        vector<int> v(N), o, c(N);
19        auto dfs = [&](auto&& self, int x) -> void {
20            v[x] = 1;
21            for (int y : g[x])
22                if (!v[y])
23                    self(self, y);
24            o.push_back(x);
25        };
26        for (int i = 0; i < N; i++)
27            if (!v[i])
28                dfs(dfs, i);
29
30        fill(v.begin(), v.end(), 0);
31        reverse(o.begin(), o.end());
32        int k = 0;
33        auto rdfs = [&](auto&& self, int x) -> void {
34            v[x] = 1;
35            c[x] = k;

```

twosat.cpp (cont.)

```

36     for (int y : rg[x])
37         if (!v[y])
38             self(self, y);
39 };
40 for (int x : o)
41     if (!v[x])
42         rdfs(rdfs, x), k++;
43
44 for (int i = 0; i < n; i++) {
45     if (c[2 * i] == c[2 * i + 1])
46         return false;
47     a[i] = c[2 * i] < c[2 * i + 1];
48 }
49 return true;
50 }
51
52 vector<bool> answer() { return a; }
53 };

```

rerooting.cpp

```

1 /*
2  <Rerooting(全方位木 DP)>
3   - n 頂点の木 T の各頂点を値とした時の木 DP の結果を得る
4
5   [実装/関数]
6   (構造体)
7   - struct RerootingDP<class E, class V, merge, e, put_edge,
  ↳ put_vertex> dp(n)
8       - E : 辺に持たせる情報の型
9       - V : 頂点に持たせる情報の型(≠頂点の番号)
10  ↳ 各頂点の dp 表にはこの型の値を記録する
11       - E merge(E e1, E e2) : 同じ頂点を親側に持つ二つの辺を
  ↳ マージして新たな辺の情報として持つ
12       - E e() : merge の単位元
13       - E put_edge(V v, int i) : 頂点 i の親側に辺を 1 つ張った時
  ↳ , その辺が持つ情報を計算
14       - V put_vertex(E e, int i) : 頂点 i の全ての子側の辺をマ
  ↳ ージした値から、頂点の情報を計算
15       - 木の葉の dp の値は、E の単位元 e によって put_vertex(e,
  ↳ i)により定まる
16       (葉 : put_vertex(e,i) により生成
17       他 : 葉の値をそれぞれ put_edge -> merge ->
  ↳ 得られた値を put_vertex)
18   (関数)

```

rerooting.cpp (cont.)

```

19     - void add_edge(int u, int v, int i, int j) :
20         頂点 u から v への番号 i の辺と, 頂点 v から u への番号 j の辺を追
↳ 加する (i=j でもよい)
21
22     - V build(int v) : 頂点 v を根とした時の dp[v] の値を返す (
↳ 1 回しか呼べない)
23
24     - vector<V> reroot() : 頂点 i を根とした時の dp[
↳ i] の値を持つ配列を返す
25         (事前に build をちょうど 1 回呼んでいる必要あり)
26
27     [計算時間]
28     - コンストラクタ 0(n), build 0(n), reroot 0(n)
29
30     [備考]
31     - (codeforces の G++ 20 では E として pii を用いるとうまく動かなか
↳ った G++ 17 を用いること)
32     - put_edge と put_vertex の添字 i の型は int で指定すること (
↳ ll とするとコンパイルエラー)
33
34     [参照]
35     - traP
36
37     [verified at]
38     - https://atcoder.jp/contests/abc394/submissions/63066845
39
40 */
41
42 /*
43     - n 頂点の木 T が与えられている
44
45     - RerootingDP<E,V,merge,e,put_edge,put_vertex> g(n) :
↳ コンストラクタ 0(n)
46         E : 親の dp の計算をする際に用いる値の型
47         V : 頂点の dp の値の型
48         E put_edge (V v, int i) : 頂点 v を根とする部分木の dp の値に辺
↳ i を追加した時に得られる値
49         E merge(E e1, E e2) : 2 つの子について求めた put_edge の値を
↳ 結合する
50         merge は可換モノイドによる結合を要求される
51         E e() : merge の単位元を返す関数
52         V put_vertex(E e, int u) : 全ての子について merge した値から
↳ , 頂点 u の dp[u] を計算する
53     - void add_edge(int u, int v, int idx, int xdi) :
↳ 頂点 u->v の辺番号 idx の辺と

```

rerooting.cpp (cont.)

```

54     頂点 v->u の辺番号 xdi の辺を追加する (ちょうど n-1 回呼ぶ, なら
↳ し 0(1))
55     - V bulid(int v) : 頂点 v を根とした時の, dp[v] の値を返す (
↳ 1 回までしか呼べない 0(n))
56     - vector<V> reroot() : 長さ n の配列 (i 番目には, 頂点 i を根とした時
↳ の, dp[i] の値) を返す
57         (先に build をちょうど 1 回呼んでいる必要あり 0(n))
58 */
59
60 template <class E, class V, E (*merge)(E, E), E (*e)(),
61     E (*put_edge)(V, int), V (*put_vertex)(E, int)>
62 struct RerootingDP {
63     struct edge {
64         int to, idx, xdi;
65     };
66     RerootingDP(int n_ = 0) : n(n_), inner_edge_id(0) {
67         es.resize(2 * n - 2);
68         start.resize(2 * n - 2);
69         if (n == 1)
70             es_build();
71     }
72     void add_edge(int u, int v, int idx, int xdi) {
73         start[inner_edge_id] = u;
74         es[inner_edge_id] = {v, idx, xdi};
75         inner_edge_id++;
76         start[inner_edge_id] = v;
77         es[inner_edge_id] = {u, xdi, idx};
78         inner_edge_id++;
79         if (inner_edge_id == 2 * n - 2) {
80             es_build();
81         }
82     }
83     vector<V> build(int root_ = 0) {
84         root = root_;
85         vector<V> subdp(n);
86         subdp[0] = put_vertex(e(), 0);
87         outs.resize(n);
88         vector<int> geta(n + 1, 0);
89         for (int i = 0; i < n; i++)
90             geta[i + 1] = start[i + 1] - start[i] - 1;
91         geta[root + 1]++;
92         for (int i = 0; i < n; i++)
93             geta[i + 1] += geta[i];

```

rerooting.cpp (cont.)

```

94  auto dfs = [&](auto sfs, int v, int f) -> void {
95      E val = e();
96      for (int i = start[v]; i < start[v + 1]; i++) {
97          if (es[i].to == f) {
98              swap(es[start[v + 1] - 1], es[i]);
99          }
100         if (es[i].to == f)
101             continue;
102         sfs(sfs, es[i].to, v);
103         E nval = put_edge(subdp[es[i].to], es[i].idx);
104         outs[geta[v]++] = nval;
105         val = merge(val, nval);
106     }
107     subdp[v] = put_vertex(val, v);
108 };
109 dfs(dfs, root, -1);
110 return subdp;
111 }
112 vector<V> reroot() {
113     vector<E> reverse_edge(n);
114     reverse_edge[root] = e();
115     vector<V> answers(n);
116     auto dfs = [&](auto sfs, int v) -> void {
117         int le = outs_start(v);
118         int ri = outs_start(v + 1);
119         int siz = ri - le;
120         vector<E> rui(siz + 1);
121         rui[siz] = e();
122         for (int i = siz - 1; i >= 0; i--) {
123             rui[i] = merge(outs[le + i], rui[i + 1]);
124         }
125         answers[v] = put_vertex(merge(rui[0], reverse_edge[v]), v);
126         E lui = e();
127         for (int i = 0; i < siz; i++) {
128             V rdp = put_vertex(
129                 merge(merge(lui, rui[i + 1]), reverse_edge[v]), v);
130             reverse_edge[es[start[v] + i].to] =
131                 put_edge(rdp, es[start[v] + i].xd);
132             lui = merge(lui, outs[le + i]);
133             sfs(sfs, es[start[v] + i].to);
134         }
135     };
136     dfs(dfs, root);
137     return answers;
138 }

```

rerooting.cpp (cont.)

```

139
140 private:
141     int n, root, inner_edge_id;
142     vector<E> outs;
143     vector<edge> es;
144     vector<int> start;
145     int outs_start(int v) {
146         int res = start[v] - v;
147         if (root < v)
148             res++;
149         return res;
150     }
151     void es_build() {
152         vector<edge> nes(2 * n - 2);
153         vector<int> nstart(n + 2, 0);
154         for (int i = 0; i < 2 * n - 2; i++)
155             nstart[start[i] + 2]++;
156         for (int i = 0; i < n; i++)
157             nstart[i + 1] += nstart[i];
158         for (int i = 0; i < 2 * n - 2; i++)
159             nes[nstart[start[i] + 1]++] = es[i];
160         swap(es, nes);
161         swap(start, nstart);
162     }
163 };

```

maxflow.cpp

```

1  template <class Cap>
2  struct mf_graph {
3      struct E {
4          int to, rev;
5          Cap cap;
6      };
7      struct edge {
8          int from, to;
9          Cap cap, flow;
10     };
11     int N;
12     vector<vector<E>> g;
13     vector<pair<int, int>> pos;
14
15     mf_graph(int n = 0) : N(n), g(n) {}
16

```

maxflow.cpp (cont.)

```

17 int add_edge(int s, int t, Cap c) {
18     int a = g[s].size(), b = g[t].size() + (s == t);
19     pos.push_back({s, a});
20     g[s].push_back({t, b, c});
21     g[t].push_back({s, a, 0});
22     return (int)pos.size() - 1;
23 }
24
25 edge get_edge(int i) {
26     auto [v, j] = pos[i];
27     auto &e = g[v][j], &r = g[e.to][e.rev];
28     return {v, e.to, e.cap + r.cap, r.cap};
29 }
30
31 vector<edge> edges() {
32     vector<edge> r;
33     for (int i = 0; i < (int)pos.size(); i++)
34         r.push_back(get_edge(i));
35     return r;
36 }
37
38 void change_edge(int i, Cap c, Cap f) {
39     auto [v, j] = pos[i];
40     auto &e = g[v][j], &r = g[e.to][e.rev];
41     e.cap = c - f;
42     r.cap = f;
43 }
44
45 Cap flow(int s, int t) {
46     return flow(s, t, numeric_limits<Cap>::max());
47 }
48
49 Cap flow(int s, int t, Cap lim) {
50     Cap ans = 0;
51     vector<int> lv(N), it(N);
52     while (ans < lim) {
53         fill(lv.begin(), lv.end(), -1);
54         queue<int> q;
55         q.push(s);
56         lv[s] = 0;
57         while (q.size()) {
58             int v = q.front();
59             q.pop();
60             for (auto& e : g[v])
61                 if (e.cap && lv[e.to] < 0)

```

maxflow.cpp (cont.)

```

62         lv[e.to] = lv[v] + 1, q.push(e.to);
63     }
64     if (lv[t] < 0)
65         break;
66     fill(it.begin(), it.end(), 0);
67
68     auto dfs = [&](auto&& self, int v, Cap f) -> Cap {
69         if (v == t)
70             return f;
71         for (int& i = it[v]; i < (int)g[v].size(); i++) {
72             E& e = g[v][i];
73             if (e.cap && lv[e.to] == lv[v] + 1)
74                 if (Cap x = self(self, e.to, min(f, e.cap))) {
75                     e.cap -= x;
76                     g[e.to][e.rev].cap += x;
77                     return x;
78                 }
79             i++;
80         }
81         return 0;
82     };
83     while (ans < lim) {
84         Cap x = dfs(dfs, s, lim - ans);
85         if (!x)
86             break;
87         ans += x;
88     }
89 }
90 return ans;
91 }
92
93 vector<bool> min_cut(int s) {
94     vector<bool> z(N);
95     queue<int> q;
96     q.push(s);
97     z[s] = 1;
98     while (q.size()) {
99         int v = q.front();
100        q.pop();
101        for (auto& e : g[v])
102            if (e.cap && !z[e.to])
103                z[e.to] = 1, q.push(e.to);
104    }

```

maxflow.cpp (cont.)

```

105     return z;
106 }
107 };

```

mincostflow.cpp

```

1  template <class Cap, class Cost>
2  struct mcf_graph {
3      struct E {
4          int to, rev;
5          Cap cap;
6          Cost cost;
7      };
8      struct edge {
9          int from, to;
10         Cap cap, flow;
11         Cost cost;
12     };
13     int N;
14     vector<vector<E>> g;
15     vector<pair<int, int>> pos;
16
17     mcf_graph(int n = 0) : N(n), g(n) {}
18
19     int add_edge(int s, int t, Cap c, Cost w) {
20         int a = g[s].size(), b = g[t].size() + (s == t);
21         pos.push_back({s, a});
22         g[s].push_back({t, b, c, w});
23         g[t].push_back({s, a, 0, -w});
24         return (int)pos.size() - 1;
25     }
26
27     edge get_edge(int i) {
28         auto [v, j] = pos[i];
29         auto &e = g[v][j], &r = g[e.to][e.rev];
30         return {v, e.to, e.cap + r.cap, r.cap, e.cost};
31     }
32
33     vector<edge> edges() {
34         vector<edge> r;
35         for (int i = 0; i < (int)pos.size(); i++)
36             r.push_back(get_edge(i));
37         return r;
38     }
39

```

mincostflow.cpp (cont.)

```

40     pair<Cap, Cost> flow(int s, int t) {
41         return flow(s, t, numeric_limits<Cap>::max());
42     }
43
44     pair<Cap, Cost> flow(int s, int t, Cap lim) {
45         return slope(s, t, lim).back();
46     }
47
48     vector<pair<Cap, Cost>> slope(int s, int t) {
49         return slope(s, t, numeric_limits<Cap>::max());
50     }
51
52     vector<pair<Cap, Cost>> slope(int s, int t, Cap lim) {
53         const Cost INF = numeric_limits<Cost>::max();
54         vector<Cost> h(N), d(N);
55         vector<int> pv(N), pe(N);
56
57         Cap f = 0;
58         Cost c = 0, last = -1;
59         vector<pair<Cap, Cost>> res{{0, 0}};
60
61         while (f < lim) {
62             fill(d.begin(), d.end(), INF);
63             d[s] = 0;
64
65             priority_queue<pair<Cost, int>, vector<pair<Cost, int>>,
66                 greater<pair<Cost, int>>>
67                 q;
68             q.push({0, s});
69
70             while (q.size()) {
71                 auto [dv, v] = q.top();
72                 q.pop();
73                 if (dv != d[v])
74                     continue;
75
76                 for (int i = 0; i < (int)g[v].size(); i++) {
77                     E& e = g[v][i];
78                     if (!e.cap)
79                         continue;
80
81                     __int128 nd = (__int128)dv + e.cost + h[v] - h[e.to];
82                     if (nd < d[e.to]) {
83                         d[e.to] = (Cost)nd;
84                         pv[e.to] = v;

```

mincostflow.cpp (cont.)

```

85         pe[e.to] = i;
86         q.push({(Cost)nd, e.to});
87     }
88 }
89 }
90
91 if (d[t] == INF)
92     break;
93
94 for (int v = 0; v < N; v++)
95     if (d[v] < INF)
96         h[v] += d[v];
97
98 Cap x = lim - f;
99 for (int v = t; v != s; v = pv[v])
100     x = min(x, g[pv[v]][pe[v]].cap);
101
102 for (int v = t; v != s; v = pv[v]) {
103     E& e = g[pv[v]][pe[v]];
104     e.cap -= x;
105     g[v][e.rev].cap += x;
106 }
107
108 Cost w = h[t] - h[s];
109 f += x;
110 c += x * w;
111
112 if (last == w)
113     res.pop_back();
114 res.push_back({f, c});
115 last = w;
116 }
117
118 return res;
119 }
120 };

```

math.cpp

```

1 using ll = long long;
2 using ull = unsigned long long;
3
4 ll smod(ll x, ll m) {
5     return (x % m) < 0 ? x + m : x;
6 }
7

```

math.cpp (cont.)

```

8 pair<ll, ll> igcd(ll a, ll b) {
9     a = smod(a, b);
10    if (!a)
11        return {b, 0};
12    ll s = b, t = a, x = 0, y = 1;
13    while (t) {
14        ll q = s / t;
15        s -= t * q;
16        x -= y * q;
17        swap(s, t);
18        swap(x, y);
19    }
20    if (x < 0)
21        x += b / s;
22    return {s, x};
23 }
24
25 ll pow_mod(ll x, ll n, int m) {
26     ll r = 1 % m;
27     x = smod(x, m);
28     for (; n; n >>= 1, x = x * x % m)
29         if (n & 1)
30             r = r * x % m;
31     return r;
32 }
33
34 ll inv_mod(ll x, ll m) {
35     return igcd(x, m).second;
36 }
37
38 pair<ll, ll> crt(const vector<ll>& r, const vector<ll>& m) {
39     ll R = 0, M = 1;
40     for (int i = 0; i < (int)r.size(); i++) {
41         ll x = smod(r[i], m[i]);
42         auto [g, im] = igcd(M, m[i]);
43         ll u = m[i] / g;
44         if ((x - R) % g)
45             return {0, 0};
46         ll t = (__int128)smod((x - R) / g, u) * im % u;
47         R += M * t;
48         M *= u;
49         R = smod(R, M);
50     }
51     return {R, M};
52 }

```

math.cpp (cont.)

```

53
54 ll floor_sum(ll n, ll m, ll a, ll b) {
55     ull z = 0;
56     if (a < 0) {
57         ull x = smod(a, m);
58         z -= 1ULL * n * (n - 1) / 2 * ((x - a) / m);
59         a = x;
60     }
61     if (b < 0) {
62         ull x = smod(b, m);
63         z -= 1ULL * n * ((x - b) / m);
64         b = x;
65     }
66     for (;;) {
67         if (a >= m)
68             z += 1ULL * n * (n - 1) / 2 * (a / m), a %= m;
69         if (b >= m)
70             z += 1ULL * n * (b / m), b %= m;
71         ull y = (ull)a * n + b;
72         if (y < (ull)m)
73             return (ll)z;
74         n = y / m;
75         b = y % m;
76         swap(a, m);
77     }
78 }

```

modint_convolution.cpp

```

1 struct mint {
2     static const int M = 998244353;
3     int x;
4     mint(long long v = 0) : x((v % M + M) % M) {}
5     int val() const { return x; }
6     static int mod() { return M; }
7     mint& operator+=(mint a) {
8         if ((x += a.x) >= M)
9             x -= M;
10        return *this;
11    }
12    mint& operator-=(mint a) {
13        if ((x -= a.x) < 0)
14            x += M;
15        return *this;
16    }

```

modint_convolution.cpp (cont.)

```

17    mint& operator*=(mint a) {
18        x = (long long)x * a.x % M;
19        return *this;
20    }
21    mint pow(long long n) const {
22        mint a = *this, r = 1;
23        for (; n; n >>= 1, a *= a)
24            if (n & 1)
25                r *= a;
26        return r;
27    }
28    mint inv() const { return pow(M - 2); }
29    mint& operator/=(mint a) { return *this *= a.inv(); }
30    friend mint operator+(mint a, mint b) { return a += b; }
31    friend mint operator-(mint a, mint b) { return a -= b; }
32    friend mint operator*(mint a, mint b) { return a *= b; }
33    friend mint operator/(mint a, mint b) { return a /= b; }
34 };
35 using modint998244353 = mint;
36 void ntt(vector<mint>& a, bool inv = 0) {
37     int n = a.size();
38     for (int i = 1, j = 0; i < n; i++) {
39         int b = n >> 1;
40         for (; j & b; b >>= 1)
41             j ^= b;
42         j ^= b;
43         if (i < j)
44             swap(a[i], a[j]);
45     }
46     for (int l = 2; l <= n; l <= 1) {
47         mint w = mint(3).pow((mint::M - 1) / l);
48         if (inv)
49             w = w.inv();
50         for (int i = 0; i < n; i += l) {
51             mint z = 1;
52             for (int j = 0; j < l / 2; j++) {
53                 mint x = a[i + j], y = a[i + j + l / 2] * z;
54                 a[i + j] = x + y;
55                 a[i + j + l / 2] = x - y;
56                 z *= w;
57             }
58         }
59     }

```

modint_convolution.cpp (cont.)

```

60 if (inv) {
61     mint z = mint(n).inv();
62     for (auto& x : a)
63         x *= z;
64 }
65 }
66 vector<mint> convolution(vector<mint> a, vector<mint> b) {
67     if (a.empty() || b.empty())
68         return {};
69     int n = a.size() + b.size() - 1, z = 1;
70     while (z < n)
71         z <= 1;
72     a.resize(z);
73     b.resize(z);
74     ntt(a);
75     ntt(b);
76     for (int i = 0; i < z; i++)
77         a[i] *= b[i];
78     ntt(a, 1);
79     a.resize(n);
80     return a;
81 }

```

geometry.cpp

```

1 using ld = long double;
2 using Point = complex<ld>;
3 using pt = Point;
4 const ld EPS = 1e-10L, PI = acos(-1.L);
5
6 inline bool equal(
7     const ld& a, const ld& b) { //a,b が誤差を許して等しいか
8     return fabsl(a - b) < EPS;
9 }
10 inline int sgn(const ld& x) { //x の符号を-1,0,1 で返す
11     return (x > EPS) - (x < -EPS);
12 }
13
14 Point unitVector(const Point& a) { //a と同方向の単位ベクトル
15     assert(abs(a) > EPS);
16     return a / abs(a);
17 }
18 Point normalVector(const Point& a) { //a を反時計回りに 90 度回転
19     return a * Point(0, 1);
20 }
21 ld dot(const Point& a, const Point& b) { //内積

```

geometry.cpp (cont.)

```

22 return real(conj(a) * b);
23 }
24 ld cross(const Point& a, const Point& b) { //外積
25     return imag(conj(a) * b);
26 }
27 Point rotate(const Point& p, ld t) { //p を反時計回りに t(rad)回転
28     return p * polar<ld>(1, t);
29 }
30 ld radianToDegree(ld x) { //rad を度に変換
31     return x * 180 / PI;
32 }
33 ld degreeToRadian(ld x) { //度を rad に変換
34     return x * PI / 180;
35 }
36
37 struct Line {
38     Point a, b;
39     Line() = default;
40     Line(Point x, Point y) : a(x), b(y) {} //2 点 x,y を通る直線(線分)
41     Line(ld A, ld B, ld C) { //Ax+By=C を表す直線
42         assert(!equal(A, 0) || !equal(B, 0));
43         if (!equal(B, 0))
44             a = {0, C / B}, b = {1, (C - A) / B};
45         else
46             a = {C / A, 0}, b = {C / A, 1};
47     }
48 };
49
50 struct Circle {
51     Point p;
52     ld r;
53     Circle() = default;
54     Circle(Point x, ld y) : p(x), r(y) {} //中心 x,半径 y の円
55 };
56
57 bool onLine(const Line& l, const Point& p) { //p が直線 l 上か
58     return fabsl(cross(l.b - l.a, p - l.a)) <= EPS;
59 }
60
61 bool onSegment(const Line& l, const Point& p) { //p が線分 l 上か
62     return ccw(l.a, l.b, p) == 0;
63 }
64
65 Circle circumCircle(
66     Point a, Point b, Point c) { //三角形 abc の外接円

```

geometry.cpp (cont.)

```

67  assert(fabsl(cross(b - a, c - a)) > EPS);
68  Point x = (a + b) / 2, y = (a + c) / 2;
69  Line l = {x, x + normalVector(b - a)};
70  Line m = {y, y + normalVector(c - a)};
71  Point p = crossPoint(l, m);
72  return {p, abs(p - a)};
73 }
74
75 Point projection(
76     const Line& l, const Point& p) { //lへpから下ろした垂線の足
77     Point d = l.b - l.a;
78     assert(abs(d) > EPS);
79     return l.a + d * dot(p - l.a, d) / norm(d);
80 }
81 Point reflection(
82     const Line& l, const Point& p) { //lに関するpの対称点
83     return 2*L * projection(l, p) - p;
84 }
85
86 int ccw(const Point& a, Point b, Point c) { //a,b,cの位置関係を判定
87     b -= a;
88     c -= a;
89     ld x = cross(b, c);
90     if (abs(x) > EPS)
91         return x > 0 ? 1 : -1; //反時計/時計
92     if (dot(b, c) < -EPS)
93         return 2; //c-a-b
94     if (norm(b) + EPS < norm(c))
95         return -2; //a-b-c
96     return 0; //cが線分ab上
97 }
98
99 bool isOrthogonal(
100     const Line& a, const Line& b) { //2直線が直交するか
101     return equal(dot(a.b - a.a, b.b - b.a), 0);
102 }
103 bool isParallel(const Line& a, const Line& b) { //2直線が平行か
104     return equal(cross(a.b - a.a, b.b - b.a), 0);
105 }
106 bool isIntersect(
107     const Line& a, const Line& b) { //2線分が交差するか
108     return ccw(a.a, a.b, b.a) * ccw(a.a, a.b, b.b) <= 0 &&
109         ccw(b.a, b.b, a.a) * ccw(b.a, b.b, a.b) <= 0;
110 }
111

```

geometry.cpp (cont.)

```

112 Point crossPoint(const Line& a, const Line& b) { //2直線の交点
113     Point d = a.b - a.a, e = b.b - b.a;
114     ld z = cross(d, e), w = cross(d, b.a - a.a);
115     if (equal(z, 0)) {
116         assert(equal(w, 0));
117         return b.a;
118     }
119     return a.a + d * cross(b.a - a.a, e) / z;
120 }
121
122 ld distanceBetweenLineAndPoint(
123     const Line& l, const Point& p) { //直線lと点pの距離
124     Point d = l.b - l.a;
125     assert(abs(d) > EPS);
126     return fabsl(cross(d, p - l.a)) / abs(d);
127 }
128 ld distanceBetweenSegmentAndPoint(
129     const Line& l, const Point& p) { //線分lと点pの距離
130     Point d = l.b - l.a;
131     assert(abs(d) > EPS);
132     if (dot(d, p - l.a) < EPS)
133         return abs(p - l.a);
134     if (dot(-d, p - l.b) < EPS)
135         return abs(p - l.b);
136     return fabsl(cross(d, p - l.a)) / abs(d);
137 }
138 ld distanceBetweenSegments(
139     const Line& a, const Line& b) { //2線分間の距離
140     if (isIntersect(a, b))
141         return 0;
142     return min({distanceBetweenSegmentAndPoint(a, b.a),
143                 distanceBetweenSegmentAndPoint(a, b.b),
144                 distanceBetweenSegmentAndPoint(b, a.a),
145                 distanceBetweenSegmentAndPoint(b, a.b)});
146 }
147
148 ld SignedPolygonArea(
149     const vector<Point>& p) { //多角形の符号付き面積
150     ld s = 0;
151     for (int i = 0, n = p.size(); i < n; i++)
152         s += cross(p[i], p[(i + 1) % n]);
153     return s / 2;
154 }
155 ld PolygonArea(const vector<Point>& p) { //多角形の面積

```

geometry.cpp (cont.)

```

156 return fabsl(SignedPolygonArea(p));
157 }
158
159 bool isConvex(const vector<Point>& p) { //多角形が凸か
160     if (p.size() < 3)
161         return false;
162     int s = 0;
163     for (int i = 0, n = p.size(); i < n; i++) {
164         int t = sgn(cross(
165             p[(i + 1) % n] - p[i], p[(i + 2) % n] - p[(i + 1) % n]));
166         if (t && s && t != s)
167             return false;
168         if (t)
169             s = t;
170     }
171     return true;
172 }
173
174 int isContained(const vector<Point>& g,
175     const Point& p) { //点 p が多角形 g の内/边上/外か(2/1/0)
176     bool in = 0;
177     for (int i = 0, n = g.size(); i < n; i++) {
178         Point a = g[i] - p, b = g[(i + 1) % n] - p;
179         if (imag(a) > imag(b))
180             swap(a, b);
181         if (imag(a) <= EPS && EPS < imag(b) && cross(a, b) < -EPS)
182             in ^= 1;
183         if (equal(cross(a, b), 0) && dot(a, b) <= EPS)
184             return 1;
185     }
186     return in ? 2 : 0;
187 }
188
189 vector<Point> ConvexHull(vector<Point> p,
190     bool col =
191     false) { //凸包を反時計回りに返す(col=trueなら边上も含む)
192     sort(p.begin(), p.end(), [](Point a, Point b) {
193         return real(a) != real(b) ? real(a) < real(b)
194             : imag(a) < imag(b);
195     });
196     p.erase(unique(p.begin(), p.end(),
197         [](Point a, Point b) { return abs(a - b) < EPS; }),
198         p.end());
199
200     int n = p.size(), k = 0;

```

geometry.cpp (cont.)

```

201 if (n <= 1)
202     return p;
203 vector<Point> h(2 * n);
204 if (col) {
205     bool all = 1;
206     for (int i = 2; i < n; i++)
207         if (fabsl(cross(p[1] - p[0], p[i] - p[0])) > EPS)
208             all = 0;
209     if (all)
210         return p;
211 }
212
213 auto bad = [&](
214     Point a, Point b, Point c) { //凸包に不要な中間点か
215     ld x = cross(b - a, c - b);
216     return col ? x < -EPS : x < EPS;
217 };
218
219 for (auto x : p) {
220     while (k >= 2 && bad(h[k - 2], h[k - 1], x))
221         --k;
222     h[k++] = x;
223 }
224 for (int i = n - 2, t = k + 1; i >= 0; i--) {
225     while (k >= t && bad(h[k - 2], h[k - 1], p[i]))
226         --k;
227     h[k++] = p[i];
228 }
229 h.resize(k - 1);
230 return h;
231 }
232
233 int isIntersect(const Circle& a,
234     const Circle& b) { //2 円の位置関係 (返り値=共通接線数, -1 は同一円)
235     ld d = abs(a.p - b.p), s = a.r + b.r, t = fabsl(a.r - b.r);
236     if (equal(d, 0) && equal(a.r, b.r))
237         return -1;
238     if (d > s + EPS)
239         return 4; //離れている
240     if (equal(d, s))
241         return 3; //外接
242     if (d < t - EPS)
243         return 0; //内包
244     if (equal(d, t))
245         return 1; //内接

```

geometry.cpp (cont.)

```

246 return 2; //2 点で交わる
247 }
248
249 Circle inCircle(const Point& a, const Point& b,
250 const Point& c) { //三角形 abc の内接円
251 ld A = abs(b - c), B = abs(a - c), C = abs(a - b);
252 assert(A + B + C > EPS && fabs1(cross(b - a, c - a)) > EPS);
253 Point p = (a * A + b * B + c * C) / (A + B + C);
254 return {p, distanceBetweenLineAndPoint({a, b}, p)};
255 }
256
257 vector<Point> crossPoint(
258 const Circle& c, const Line& l) { //円 c と直線 l の交点
259 ld d = distanceBetweenLineAndPoint(l, c.p);
260 if (d > c.r + EPS)
261     return {};
262 Point h = projection(l, c.p);
263 if (equal(d, c.r))
264     return {h};
265 Point u =
266     unitVector(l.b - l.a) * sqrt(max<ld>(0, c.r * c.r - d * d));
267 return {h - u, h + u};
268 }
269
270 vector<Point> crossPoint(
271 const Circle& a, const Circle& b) { //2 円の交点
272 int m = isIntersect(a, b);
273 if (m == 4 || m == 0 || m == -1)
274     return {};
275
276 ld d = abs(b.p - a.p);
277 ld x = (a.r * a.r + d * d - b.r * b.r) / (2 * d);
278 ld y = sqrt(max<ld>(0, a.r * a.r - x * x));
279
280 Point u = (b.p - a.p) / d, h = a.p + u * x;
281 if (m == 1 || m == 3)
282     return {h};
283
284 Point v = u * Point(0, y);
285 return {h + v, h - v};
286 }
287
288 vector<Point> tangentToCircle(
289 const Point& p, const Circle& c) { //点 p から円 c への接点
290 ld d = norm(c.p - p);

```

geometry.cpp (cont.)

```

291 if (d < c.r * c.r - EPS)
292     return {};
293 return crossPoint(c, Circle(p, sqrt(max<ld>(0, d - c.r * c.r))));
294 }
295
296 vector<Line> tangent(
297 const Circle& a, const Circle& b) { //2 円の共通接線
298 vector<Line> r;
299 ld g = abs(a.p - b.p);
300 if (equal(g, 0))
301     return r;
302
303 Point u = unitVector(b.p - a.p), v = normalVector(u);
304
305 for (int s : {-1, 1}) {
306     ld h = (a.r + b.r * s) / g, q = 1 - h * h;
307     if (q < -EPS)
308         continue;
309
310     if (fabs1(q) < EPS) {
311         Point w = h > 0 ? u : -u;
312         r.push_back({a.p + w * a.r, a.p + w * a.r + v});
313     } else {
314         Point U = u * h, V = v * sqrt(max<ld>(0, q));
315         for (int t : {1, -1}) {
316             Point w = U + V * (ld)t;
317             r.push_back({a.p + w * a.r, b.p - w * (b.r * s)});
318         }
319     }
320 }
321 return r;
322 }

```